

# 8051 Assembly Language Questions & Solutions

## 1 Array Search Problems

### Problem 1: Search for a Byte in an Array

Write an assembly language program to find whether a given byte is available in the given sequence or not.

- If the byte is **found**, write FFH in R3.
- If the byte is **not found**, write 00H in R3.

### Solution

**Given:** An array of numbers stored in data memory at locations {20H, 21H, 22H, 23H} with contents [15, 04, 06, 45]. We need to search for the value 45H.

```
1 ; Program for finding a number in array of numbers
2 ; Array stored in memory locations {20H, 21H, 22H, 23H}
3 ; Contents: [15, 04, 06, 45]
4
5     ORG 0000H           ; Sets program start address at 0000H
6
7     MOV R0, #20H        ; R0 = starting address of array
8     MOV R3, #00H        ; R3 = search result flag
9                           ; R3 = 00H means number not found
10                          ; R3 = FFH means number found
11     MOV R1, #04H        ; R1 = counter (4 elements in array)
12
13 AGAIN: MOV A, @R0        ; Load content of memory location
14                          ; pointed by R0 into accumulator
15     CJNE A, #45H, NEXT  ; Compare with 45H, if not equal
16                          ; jump to NEXT
17     MOV R3, #0FFH       ; Found! Store FFH in R3
18     SJMP DONE           ; Skip to end
19
20 NEXT: INC R0            ; Point to next memory location
21     DJNZ R1, AGAIN      ; Decrement counter, loop if not zero
22
23 DONE: END
```

## Step-by-Step Explanation

### Initialization:

- R0 = 20H: Pointer to current array element.
- R3 = 00H: Result flag (initially "not found").
- R1 = 04H: Loop counter (4 elements).

### Search Loop (AGAIN):

1. MOV A, @R0: Load value at address in R0 into accumulator.
2. CJNE A, #45H, NEXT: Compare A with 45H.
  - If not equal → jump to NEXT.
  - If equal → continue to next instruction.
3. MOV R3, #0FFH: Set flag to "found".
4. SJMP DONE: Exit loop immediately.

### Not Match (NEXT):

1. INC R0: Move to next array element.
2. DJNZ R1, AGAIN: Decrement counter, loop back if not zero.

### Key Instructions Used:

| Instruction | Description                                                |
|-------------|------------------------------------------------------------|
| MOV A, @R0  | Indirect addressing – load value from address stored in R0 |
| CJNE        | Compare and Jump if Not Equal                              |
| SJMP        | Short Jump (unconditional, 2-byte instruction)             |
| DJNZ        | Decrement and Jump if Not Zero                             |
| INC         | Increment register by 1                                    |

## 1.1 Memory State Example

| Address | Value | Check                         |
|---------|-------|-------------------------------|
| 20H     | 15H   | ≠ 45H, continue               |
| 21H     | 04H   | ≠ 45H, continue               |
| 22H     | 06H   | ≠ 45H, continue               |
| 23H     | 45H   | = 45H, <b>FOUND!</b> R3 = FFH |

## 2 Bit Manipulation Problems

### Problem 2: Count 1s and 0s in an 8-bit Number

Write an assembly language program to count the number of 1s and 0s in a given 8-bit number.

### Solution

```
1 ; Program to compute number of 1s and 0s in 8-bit number
2 ; Logic: Rotate A through carry 8 times.
3 ; If C=1, increment R1 (1s). If C=0, increment R2 (0s).
4
5     ORG 0000H           ; Set program start address
6
7     MOV A, #05H         ; Input number: 05H (00000101B)
8     MOV R1, #00H        ; R1 = Counter for 1s
9     MOV R2, #00H        ; R2 = Counter for 0s
10    MOV R3, #08H        ; R3 = Total bits counter (8 bits)
11    CLR C               ; Clear carry flag
12
13 UP:   RRC A             ; Rotate Accumulator right through carry
14       JNC DOWN         ; If C = 0, jump to DOWN
15       INC R1           ; If C = 1, increment 1s counter
16       SJMP EXIT       ; Skip DOWN logic
17
18 DOWN: INC R2           ; Increment 0s counter
19
20 EXIT: DJNZ R3, UP      ; Repeat for all 8 bits
21
22     ; Store results in RAM
23     MOV R0, #40H       ; Point to RAM location 40H
24     MOV A, R1
25     MOV @R0, A         ; Store count of 1s at 40H
26     INC R0             ; Point to 41H
27     MOV A, R2
28     MOV @R0, A         ; Store count of 0s at 41H
29
30     END
```

### Execution Logic

- **Input:** The number 05H (Binary: 00000101).
- **Process:** The RRC A instruction shifts each bit into the Carry flag one by one.
- **Decision:**
  - If C=1, R1 is incremented.
  - If C=0 (JNC triggers), R2 is incremented.
- **Output:** After 8 iterations, R1 will contain the number of set bits (1s) and R2 will contain the number of clear bits (0s). These are stored at RAM locations 40H and 41H.

## 3 Arithmetic Operations

### Problem 3: Compute $x$ to the power $n$

Write an assembly language program to compute  $x^n$  where both  $x$  and  $n$  are 8-bit numbers and the result does not exceed 16 bits.

### Solution

```
1 ; Logic: Multiply x by itself n-1 times.
2 ; For  $x^3$ : result = x * x * x.
3
4     ORG 0000H
5
6     MOV A, #02H      ; Base x = 2
7     MOV B, #03H      ; Exponent n = 3
8
9     MOV R0, B        ; R0 = Exponent counter
10    MOV R1, A         ; R1 = Base value
11    MOV R2, #01H     ; R2 = Initialize result to 1
12
13 LOOP1: MOV A, R2      ; Load current result
14        MOV B, R1      ; Load base x
15        MUL AB         ; Multiply: A = result * x
16        MOV R2, A      ; Update lower byte of result
17        ; In this logic, A holds the cumulative product.
18
19        DJNZ R0, LOOP1 ; Decrement n, repeat until R0=0
20
21        MOV A, R2      ; Final result in Accumulator
22        END
```

## 4 Matrix Operations

### Problem 4: $2 \times 2$ Matrix Addition

Write an assembly language program to perform addition of two  $2 \times 2$  matrices.

### Solution

```
1 ; Matrix A: [5, 6; 7, 8] at 20H-23H
2 ; Matrix B: [3, 2; 1, 0] at 30H-33H
3 ; Result C = A + B stored back in 20H-23H
4
5     ORG 0000H
6
7     MOV R0, #20H    ; Pointer for Matrix A (and Result)
8     MOV R1, #30H    ; Pointer for Matrix B
9     MOV R4, #04H    ; Counter = 4 elements
10
11 AGAIN: MOV A, @R0    ; Load element from A
12         MOV R3, A    ; Store in R3
13         MOV A, @R1    ; Load element from B
14         ADD A, R3     ; Add: A = elementA + elementB
15
16         MOV @R0, A    ; Store result back in Matrix A location
17
18         INC R0        ; Increment pointers
19         INC R1
20         DJNZ R4, AGAIN ; Repeat for all 4 elements
21
22     END
```

### Problem 5: Transpose of a $2 \times 2$ Matrix

Write an assembly language program for finding the transpose of a  $2 \times 2$  matrix.

## Solution

```
1 ; Matrix A: [A00, A01; A10, A11] at 20H-23H
2 ; Result B: [B00, B01; B10, B11] at 30H-33H
3 ; Transpose: B00=A00, B01=A10, B10=A01, B11=A11
4
5     ORG 0000H
6     MOV R0, #20H      ; Source Pointer (Matrix A)
7     MOV R1, #30H      ; Destination Pointer (Matrix B)
8
9     ; A00 to B00
10    MOV A, @R0         ; A00 (20H)
11    MOV @R1, A         ; B00 (30H)
12
13    ; A01 to B10
14    INC R0              ; Point to A01 (21H)
15    INC R1
16    INC R1              ; Point to B10 (32H)
17    MOV A, @R0
18    MOV @R1, A
19
20    ; A10 to B01
21    INC R0              ; Point to A10 (22H)
22    DEC R1              ; Point to B01 (31H)
23    MOV A, @R0
24    MOV @R1, A
25
26    ; A11 to B11
27    INC R0              ; Point to A11 (23H)
28    INC R1
29    INC R1              ; Point to B11 (33H)
30    MOV A, @R0
31    MOV @R1, A
32
33    END
```

## Transpose Logic

- **Matrix Structure:** A  $2 \times 2$  matrix is stored sequentially. For  $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ , addresses are:  $a \rightarrow 20H, b \rightarrow 21H, c \rightarrow 22H, d \rightarrow 23H$ .
- **Transpose Result:**  $A^T = \begin{bmatrix} a & c \\ b & d \end{bmatrix}$ . Addresses:  $a \rightarrow 30H, c \rightarrow 31H, b \rightarrow 32H, d \rightarrow 33H$ .
- **Pointer Movement:** We manually adjust R1 to place the elements in their transposed positions.

## 5 Numerical Algorithms

### Problem 6: Square Root of an 8-bit Number

Write an assembly language program for computing the square root of an 8-bit number using an iterative technique.

### Solution

```
1 ; Iterative formula:  $x_{j+1} = (x_j + N/x_j) / 2$ 
2 ; Continue until  $x_{j+1}$  approx equals  $x_j$ 
3
4 N1 EQU 40H      ; Address of input number
5 N  EQU 41H      ; Address for result
6
7      ORG 0000H
8      MOV N1, #0FH ; Input number = 15 (approx root = 3)
9      MOV B, #01H  ; Initial guess  $x_0 = 1$ 
10     MOV R1, B     ; R1 = Current guess ( $x_j$ )
11
12 TRY:  MOV A, N1    ; A = N
13       MOV B, R1    ; B =  $x_j$ 
14       DIV AB       ; A =  $N / x_j$ 
15
16       ADD A, R1     ; A =  $(N / x_j) + x_j$ 
17       CLR C
18       RRC A        ; A =  $((N / x_j) + x_j) / 2$ 
19
20       MOV R2, A     ; R2 = New guess ( $x_{j+1}$ )
21
22       ; Check if convergence reached ( $x_{j+1} - x_j = 0$ )
23       SUBB A, R1    ; A =  $x_{j+1} - x_j$ 
24       JZ OVER       ; If difference is 0, done
25       JNC CONTINUE  ; If  $x_{j+1} > x_j$ , continue
26
27       ; If  $x_{j+1} < x_j$ , we checks if it's the closest root
28       SJMP OVER
29
30 CONTINUE:
31       MOV A, R2     ; Update  $x_j$  with  $x_{j+1}$ 
32       MOV R1, A
33       SJMP TRY      ; Next iteration
34
35 OVER:  MOV N, R2     ; Store final result
36       END
```

*Programming logic is the key to mastering 8051 assembly language.*